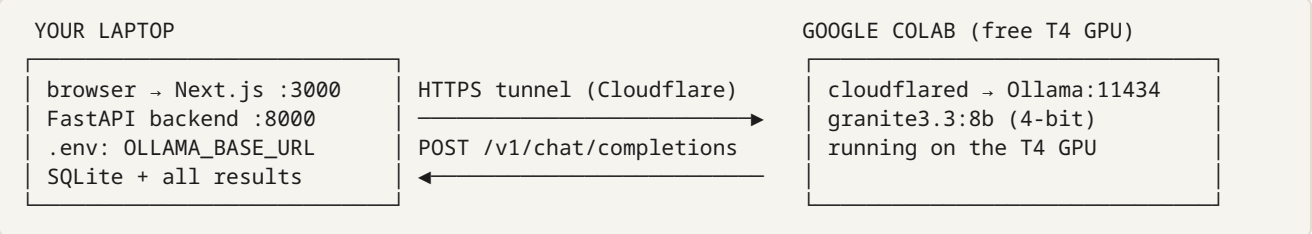
Running a 7–10B model for RedGuard on Google Colab

Goal. Your laptop (i3 10th gen, 8 GB RAM, no GPU) cannot run a 7–10B model. Google Colab gives you a free GPU for a few hours. You start a model server there, open a tunnel to it, and point RedGuard (running on your laptop) at the tunnel instead of `http://localhost:11434`. RedGuard does not need to change: both modes already talk to Ollama through one environment variable, `OLLAMA_BASE_URL`.



Only the **model** runs on Colab. The probes, the tools, the filter, the scoring, the database and every result stay on your laptop. A Colab restart loses nothing except the tunnel URL.

> Files for this guide: the notebook `docs/colab/redguard_ollama_colab.ipynb` (upload it to Colab) and this page. Nothing in either was run by the person who wrote them; test the whole path once with the single-case check in section 6 before you plan any long run. Numbers marked *(est.)* are estimates, not measurements.

---

1. What this can and cannot do

Item	Detail
Can	Serve a 4-bit 7–10B model fast enough for RedGuard (tens of tokens per second on a T4 *(est.)*), through the same OpenAI-compatible endpoint Ollama already offers. Costs nothing on the free tier.
Cannot	Stay up forever. Free Colab ends a session after a few hours at most, sooner if the tab is idle, and a GPU is not guaranteed when you ask for one *(check the current Colab FAQ for exact limits)*. Every restart gives a **new** tunnel URL.
Cannot	Fit a 7–8B model in full 16-bit precision (about 15–16 GB). 4-bit versions (what Ollama downloads by default) need about 5–6 GB.
Changes the science	A different model is a different instrument. Results from `granite3.3:8b` are **not** comparable with the earlier `granite3.3:2b` numbers. Record the model in every pre-registration (section 7).

---

2. Choose the model

Ollama tags below are the ones I know from its library; confirm each with `ollama pull <tag>` (a wrong tag fails immediately) and on https://ollama.com/library. Sizes are approximate downloads for the default 4-bit build.

Tag	Size	Why you might pick it
`granite3.3:8b` (recommended)	~5 GB	Same family and vendor as the `granite3.3:2b` you have used, so "what does scale do inside one family?" becomes a fair question. Apache-licensed.
`qwen2.5:7b`	~4.7 GB	Strong at following JSON and tool formats, which Lab mode needs.
`llama3.1:8b`	~4.9 GB	The most widely used open 8B; good for comparison with other papers.
`mistral:7b`	~4.4 GB	Small and fast.
`gemma2:9b`	~5.4 GB	Upper end of your 7–10B range.
`qwen3:8b`	~5.2 GB	A "thinking" model: it writes long reasoning before answering, which is slow and uses a lot of the token budget. RedGuard reads the `reasoning` field, but expect timeouts. Avoid for a first run.

Rule of thumb. Pick one model and use it for the whole study. Two models are a comparison (and need their own pre-registration); one model run twice is a replication.

The Lab gate. Lab mode needs a model that emits a valid JSON action almost every step. RedGuard checks this: at least 90% valid replies, at least 70% of finished pairs valid, and (in natural exposure) the entry agent reading the planted location in at least half the runs that had the chance. If the model fails a gate, the report says its rates are not comparable. A 7–8B model is much more likely to pass than the 2B was.

---

3. Before you start

1. A Google account and a browser.
2. RedGuard installed on the laptop (`npm run dev:all` works with `granite3.3:2b` or without any model).
3. Nothing else: no credit card, no Ollama on the laptop, no API key. (A free Cloudflare quick tunnel needs no account. ngrok, the alternative in section 9, needs a free account.)

---

4. Step by step

4.1 Open the notebook with a GPU

1. Go to https://colab.research.google.com → **File** → **Upload notebook** → choose `docs/colab/redguard_ollama_colab.ipynb`.

2. **Runtime → Change runtime type → T4 GPU → Save.** If it says no GPU is available, try again later or at another time of day.
3. Run the cells in order with **Shift+Enter**. What each does:

Cell	What it does	Takes
1	`nvidia-smi`: confirms a Tesla T4 with ~15 GB. If it prints nothing, you have no GPU: go back to step 2.	seconds
2	Installs Ollama (needs `zstd`, `pciutils`, `lshw` so the installer detects the GPU).	1–2 min
3	Sets `MODEL` and starts `ollama serve` with the settings below.	seconds
4	`ollama pull` the model, then sends one request so it is loaded into GPU memory. `ollama ps` should say **100% GPU**.	2–5 min *(est.)*
5	Downloads `cloudflared` and opens a quick tunnel; prints `https://<random>.trycloudflare.com`.	seconds
6	Sends a test chat request **through the tunnel** and prints the four lines to copy into `.env`.	seconds
7	(Run any time) GPU use, loaded model, last server log lines.	seconds
8	Prints the Ollama version and the model's digest: record it.	seconds
9	Stops the tunnel and server.	seconds

Server settings and why:

Setting	Value	Reason
`OLLAMA_HOST`	`127.0.0.1:11434`	Only the tunnel on the same VM can reach it; nothing listens on the open network.
`OLLAMA_KEEP_ALIVE`	`24h`	The model stays loaded; no 1-minute reload between probes.
`OLLAMA_NUM_PARALLEL`	`1`	One request at a time: results do not depend on what else is running, and memory stays low.
`OLLAMA_CONTEXT_LENGTH`	`8192`	Lab prompts grow with each step; the default is smaller. Ignored by very old Ollama versions; the installer fetches the latest.
`OLLAMA_FLASH_ATTENTION`	`1`	Less memory for the same context.
`--http-host-header localhost:11434`		Makes the request look local to Ollama; some versions reject unknown host names.

4.2 Point RedGuard at the tunnel

Edit the `.env` file in the RedGuard folder (copy `.env.example` if you have none) with the four lines cell 6 printed:

```
OLLAMA_BASE_URL=https://<random-words>.trycloudflare.com   # no trailing slash, no /v1: RedGuard
adds /v1
OLLAMA_MODEL=granite3.3:8b      # Scanner mode's Ollama agent
LAB_MODEL=granite3.3:8b         # Lab mode, when an experiment does not name a model
OLLAMA_TIMEOUT=120              # seconds per model call (Scanner agent; default 60)
```

Then restart the backend so it re-reads `.env`: stop `npm run dev:all` (Ctrl+C) and start it again.

Every time you restart the Colab notebook the URL changes: edit `OLLAMA_BASE_URL` and restart the backend again.

4.3 Check the connection from the laptop (no RedGuard needed)

```
curl -s "$OLLAMA_BASE_URL/api/tags" | head -c 300          # lists the model: proves the tunnel
and server
curl -s "$OLLAMA_BASE_URL/v1/chat/completions" -H 'Content-Type: application/json' \
  -d '{"model":"granite3.3:8b","messages":[{"role":"user","content":"Reply:
ready"}],"max_tokens":8}'
```

If both answer, RedGuard will too.

4.4 Use it

- **Scanner mode.** `/runs` → choose the agent **Ollama Local Agent** (its name still says "Local"; it now means "the Ollama server in `OLLAMA_BASE_URL`"). The session records the model, temperature and seed. For a repeat, set `SCANNER_SEED` (a different value per run) in `.env`.
- **Lab mode.** `/lab` → provider **ollama**, model `granite3.3:8b` (or leave it blank to use `LAB_MODEL`). Command line: `cd backend && python -m lab_mode.sweep --provider ollama --model granite3.3:8b …` (`docs/evaluation_protocol.md` section 4 has the commands).

---

5. Sessions end: plan for it

- **Resumable work.** A Lab sweep stores each finished pair; if Colab dies, restart the notebook, update `OLLAMA_BASE_URL`, and run `python -m lab_mode.sweep --resume <experiment id>`. It continues from the next pair with the *stored* model and settings. Scanner scans have no resume: a scan interrupted half way is marked failed and must be re-run, so start them when you know you have an hour or two.
- **Keep the tab open and awake.** Do not use scripts that click the page to fake activity: Colab's terms prohibit working around idle limits, and it can get your account restricted.
- **How long will it take?** Measure instead of guessing: run **one** Lab case (the model check in the Lab page, or `sweep --cases RGM-101 --reps 1`) and time it. Each Lab pair is two runs of up to three agents × 10 steps; an 8B model on a T4 might take a few minutes per pair *(est.)*, so 30 test cases × 5 repetitions (150 pairs) is many hours: split it over several sessions, or use fewer repetitions, or a paid tier (Colab Pro has longer sessions and faster GPUs).
- **Free tier is shared.** If you get disconnected often, run in the morning/night or consider Colab Pro.

---

6. First test, in this order (so you find problems cheaply)

1. Cells 1–6 all succeed; cell 6 prints a reply and the time of one request (a few seconds after warm-up).
2. The two `curl` commands in 4.3 work from your laptop.

3. From the RedGuard folder: `source backend/.venv/bin/activate && python -m core.lab_mode.smoke --provider ollama --model granite3.3:8b` (one model call per step; this is the first RedGuard command that uses the model).
4. A Lab **model check** from the UI (one case, one pair). Read the report's *Harness suitability* section: it says whether the model produced valid actions.
5. A Scanner scan of a small selection only after the above.

---

7. Reproducibility: what to write down

Colab changes the machine, not the method, but a result has to say what produced it. In each pre-registration page (`docs/evaluation_protocol.md` section 2) add:

- model tag **and digest** (cell 8 prints `digest` from `/api/tags`), and that it is the 4-bit build Ollama pulled;
- `ollama --version` (cell 2/8) and `OLLAMA_CONTEXT_LENGTH`;
- GPU (T4) and date. GPU arithmetic can differ in the last bits from CPU or another GPU, so a rerun on a different machine may not repeat a token-for-token result even with the same seed. Say "reproducible in distribution", not "byte-identical";
- `temperature` and the base `seed` (the Lab page and `sweep --seed` take them; Scanner uses `SCANNER_SEED`).

RedGuard already stores provider, model, temperature and seed on every session and experiment; the tunnel URL is not stored.

---

8. Safety and terms

- The tunnel URL is **public and has no password**. Anyone who learns it can use your GPU and read what you send. Treat it like a password; do not paste it in screenshots or chat; stop the notebook (cell 9) when done.
- What goes through the tunnel is RedGuard's synthetic prompts and tool results (fake secrets, canary tokens). Do not point RedGuard at real credentials or personal data while using a third-party tunnel.
- Colab's terms restrict some uses of notebooks (it is meant for interactive compute, not for serving a web service to others). Using a tunnel to reach your **own** model for your own experiment is common, but read the current Colab FAQ/terms yourself; if Google restricts it, section 9 lists other ways. *(I could not check the current wording from this machine: TODO(verify).)*
- Ollama on Colab has no authentication. Do not leave a notebook running unattended.

---

9. If something goes wrong

Symptom	Likely cause	Fix
Cell 1 prints nothing / "no GPU"	Runtime has no accelerator	Runtime → Change runtime type → T4 GPU; reconnect; try later

Symptom	Likely cause	Fix
Ollama says it runs on CPU (`ollama ps` shows `100% CPU`)	The installer did not see the GPU	Re-run cell 2 (needs `pciutils`, `lshw`), restart the runtime, re-run from cell 3
`ollama pull` "unknown model" / 404	Wrong tag	Check the name on ollama.com/library
`model requires more system memory` / out-of-memory	Model + context too big	Pick a smaller model or lower `CONTEXT` (e.g. 4096)
First request takes minutes	The model was loading	Run cell 4's warm-up; keep `OLLAMA_KEEP_ALIVE=24h`
RedGuard: `Connection refused` / timeouts	Tunnel URL changed or Colab session ended	Re-run cells 3–6, put the new URL in `.env`, restart the backend
HTTP **524** or "timeout" after ~100 s	A Cloudflare quick tunnel closes a request after about 100 s *(est.; check)*	Use a faster/smaller model, shorter `max_tokens`, or the ngrok alternative
HTTP 403 / "invalid host"	Ollama rejected the Host header	Keep `--http-host-header localhost:11434` in cell 5
Many "invalid action" replies in a Lab report	The model cannot follow the action format, or the context is cut	Check the suitability gates; raise `CONTEXT`; try another model; do not rank a model that fails the gate
Scanner agent: `Request timed out`	`OLLAMA_TIMEOUT` too small for a slow answer	Raise it (120–180) and restart the backend
Disconnected after a while	Free-tier limits	Restart, update the URL, `sweep --resume`

Other places to run the same server (if Colab will not do): Kaggle notebooks (free GPU hours, internet must be enabled in the notebook settings *(check)*), a paid Colab plan, or a rented GPU (RunPod, Vast.ai, Lambda). The recipe is the same: install Ollama, `ollama pull`, expose port 11434 through a tunnel or the provider's port-forward, set `OLLAMA_BASE_URL`. **ngrok** instead of Cloudflare: create a free account, copy your authtoken, then `pip install pyngrok`, `ngrok.set_auth_token("<token>")`, `ngrok.connect(11434, host_header="localhost:11434")` and use the printed URL *(syntax from memory: TODO(verify))*; it has no 100-second request limit.

Option B: run RedGuard itself on Colab. Clone the repository in a cell, install `backend/requirements.txt`, run `prisma generate`, and run the Lab sweep CLI there with `OLLAMA_BASE_URL=http://127.0.0.1:11434` (no tunnel); download the SQLite file at the end. It avoids the tunnel but means maintaining a second install. Only worth it for long unattended Lab sweeps.

---

10. Checklist

- [ ] T4 GPU on; cells 1–6 green; the test request through the tunnel answered.
- [ ] `.env` has `OLLAMA_BASE_URL`, `OLLAMA_MODEL`, `LAB_MODEL`, `OLLAMA_TIMEOUT`; backend restarted.
- [ ] `curl $OLLAMA_BASE_URL/api/tags` shows the model.
- [ ] One Lab model check passes the suitability gates; one small Scanner scan completes.

- [ ] Model tag, digest, Ollama version, context length, GPU and date written in the pre-registration page.
- [ ] When finished: cell 9, **Disconnect and delete runtime**, and remove the old URL from `.env`.